Ms. Nova

10 Artificial Intelligence Topics Long-form Test Knowledge Base

An original, searchable reference corpus for testing AI-powered tutoring, PDF ingestion, chunk retrieval, grounding, flashcards, concept maps, and revision quizzes.

Includes ten independent AI chapters, each over 500 words, with definitions, methodology, practical examples, evaluation, and limitations.

Recommended use: upload this PDF into Ms. Nova, then ask topic-specific questions and inspect the evidence returned by the Learning Lab.

Contents

01 | Machine Learning Fundamentals
02 | Neural Networks and Backpropagation
03 | Computer Vision and Convolutional Neural Networks
04 | Natural Language Processing and Word Embeddings
05 | Transformers, Attention, and Large Language Models
06 | Reinforcement Learning and Adaptive Tutoring
07 | Retrieval-Augmented Generation and Hybrid Search
08 | Generative AI: GANs, VAEs, and Diffusion Models
09 | Responsible AI, Fairness, and Explainability
10 | AI Agents, Planning, Tools, and Evaluation

A separate test-question sheet is provided in the download set. It is not included in this knowledge-base PDF to avoid leaking expected answers into retrieval results.

01 | Machine Learning Fundamentals

Machine learning is a branch of artificial intelligence in which a computer system learns patterns from examples rather than relying exclusively on handwritten rules. In conventional programming, a developer specifies an exact procedure and supplies input data to obtain an output. In machine learning, the developer instead supplies data, a learning objective, and a model family; a training algorithm adjusts model parameters so the system can make useful predictions on unfamiliar cases. A simple example is email spam classification. Rather than writing a rule for every suspicious phrase, an engineer provides labeled spam and non-spam messages. A model learns how words, sender patterns, and message structure correlate with the label. The central challenge is generalization: a model must work on new emails, not merely memorize messages from its training set.

Machine learning is commonly divided into supervised, unsupervised, and self-supervised learning. Supervised learning uses examples with desired outputs, such as house prices, credit-risk labels, or disease categories. A regression model predicts a continuous value, whereas a classification model predicts a category or category probabilities. Unsupervised learning searches for structure without target labels. Clustering may group customers with similar purchasing behavior, while dimensionality reduction can summarize a high-dimensional dataset for visualization. Self-supervised learning creates training signals from the data itself; predicting masked words in a sentence is one example. Reinforcement learning is another major paradigm, but it focuses on decisions and rewards over sequences of interactions rather than ordinary static input-output pairs.

An effective machine-learning workflow begins by translating a broad objective into a measurable task. Suppose a school wants to predict which learning resources students are most likely to need. The team must decide whether this is a ranking problem, classification task, or recommendation problem. It then collects relevant data, documents the origin and consent status of the records, and checks for missing values or duplicates. Numeric features may need scaling; categorical values may need encoding. A dataset is usually separated into training, validation, and test partitions. Training fits the model, validation supports decisions such as model selection, and the test partition provides a final evaluation that should not influence training decisions. When possible, a time-based split better represents future deployment than a random split.

A crucial concept is data leakage. Leakage occurs when the model sees information during development that would not actually be available when making a prediction. For example, predicting whether a student will pass an exam using the final exam score as an input makes the evaluation meaningless. Leakage can also occur when a preprocessing operation, such as feature normalization, is fitted on the full dataset before splitting. Proper pipelines fit transformations on the training partition only and then apply the learned transformations to validation and test data. This protects the evaluation from accidental hints about future examples. Class imbalance creates another difficulty: a classifier that predicts the majority class for every case can achieve high accuracy while failing at the minority cases of greatest interest.

Different metrics answer different questions. For classification, accuracy measures overall correctness, precision asks how many predicted positive cases are truly positive, and recall asks how many true positive cases were detected. The F1 score balances precision and recall with their harmonic mean. Confusion matrices show counts of true positives, false positives, true negatives, and false negatives.

For regression, mean absolute error is often easy to interpret because it describes average absolute prediction error in the target's units. Root mean squared error emphasizes larger mistakes. The choice of metric should follow the costs of mistakes, not convenience. In a medical triage example, missing a serious case may be much worse than sending one additional person for a check.

The bias-variance tradeoff explains why both overly simple and overly flexible models can fail. Underfitting happens when a model cannot capture important relationships even in training data. Overfitting happens when a model fits training details or noise but performs poorly on unseen cases. Regularization discourages overly complex solutions, cross-validation estimates performance across multiple data partitions, and early stopping halts training when validation performance stops improving. Decision trees, logistic regression, random forests, gradient-boosted trees, support vector machines, and neural networks each bring different assumptions. There is no universally best model: tabular business records often favor tree-based approaches, while images, audio, and language frequently benefit from deep learning.

For a concrete teaching example, imagine a model predicts whether a practice question will be answered correctly. Inputs might include recent performance, topic difficulty, previous hint use, and time since the last revision. The prediction should be interpreted as an estimate under the training conditions, not proof that the student knows the topic. A predicted probability of 0.80 means that similar cases were expected to be correct about eighty percent of the time if the model is calibrated. Calibration can be checked by comparing predicted probabilities with observed frequencies. This distinction is particularly useful in intelligent tutoring: prediction can inform an instructional decision, but teachers and students still need explanations, uncertainty, and opportunities to correct mistakes. Machine learning is most useful when careful problem definition, trustworthy data, meaningful evaluation, and responsible deployment are considered together.

02 | Neural Networks and Backpropagation

Artificial neural networks are flexible mathematical models composed of connected processing units often called neurons. A basic artificial neuron receives a vector of inputs, multiplies each input by a learned weight, adds a bias, and passes the result through an activation function. In symbolic form, a neuron computes an activation from a weighted sum such as z equals w dot x plus b. The activation function allows the network to represent nonlinear relationships that a simple linear model cannot capture. Popular activations include ReLU, which returns zero for negative input and the input itself for positive input, and sigmoid, which maps values into a range between zero and one. A network stacks these operations into layers so that later layers can build on features extracted by earlier ones.

A feedforward network passes information from the input layer through one or more hidden layers to an output layer. Consider predicting the price of a used car from mileage, age, location, and model type. The input features are encoded numerically and passed through hidden layers. Hidden units can learn combinations such as how age and mileage interact, while the output layer produces a predicted price. For multiclass classification, a softmax output turns raw scores into probabilities that sum to one. For binary classification, a sigmoid output is common. The exact architecture depends on the data, available computing resources, and how complex the input-output relationship is. Increasing the number of layers and neurons increases capacity but can also increase data requirements, computation, and overfitting risk.

Training a neural network means selecting weights and biases that minimize a loss function. The loss measures the disagreement between model predictions and target outputs. Mean squared error is common for regression; cross-entropy is common for classification. During a forward pass, the current network parameters produce predictions and a loss. Backpropagation then applies the chain rule of calculus to determine how small changes in each parameter would affect that loss. The resulting gradients are passed to an optimizer, such as stochastic gradient descent or Adam, which updates the parameters. This cycle repeats across batches for multiple epochs. Backpropagation is not a special form of biological learning; it is an efficient technique for computing derivatives through a chain of mathematical operations.

An epoch is one full pass through the training dataset, while a batch is a smaller group of examples processed together. Batch size influences memory use, gradient noise, and training speed. Learning rate controls the size of optimizer updates. A very high learning rate may cause unstable training or divergence; a very low learning rate may make progress unnecessarily slow. Learning-rate schedules gradually change the step size as training progresses. Validation loss helps detect overfitting: training loss may continue falling even after the network starts performing worse on unseen examples. Early stopping monitors a validation metric and halts training if improvements stop for a configured patience period. Dropout and weight decay are two regularization methods that reduce a network's tendency to memorize training examples.

Networks can encounter vanishing and exploding gradients. If derivatives repeatedly shrink while propagating through many layers, early layers receive weak learning signals. If gradients repeatedly grow, parameter updates become unstable. Activation choices, careful initialization, normalization, residual connections, and gradient clipping can help. Batch normalization standardizes intermediate activations using training statistics, while layer normalization operates across features within an

example. Residual connections add a shortcut around a transformation so deeper networks can preserve and refine existing representations. These mechanisms do not eliminate every training problem, but they make many architectures easier to optimize. Modern network design often focuses as much on stable optimization and data quality as on simply adding more layers.

A neural network's internal parameters can be difficult to interpret. Saliency methods estimate which input features most influenced an output, but such explanations require caution because they can change under small perturbations. A model can also learn harmful shortcuts, such as identifying an object by a background pattern rather than the object itself. Robust evaluation should include unfamiliar examples and relevant subgroups, and important applications may require human review. Model predictions can additionally become unreliable when real-world data differs from training data; this is called distribution shift. Monitoring after deployment is necessary because accuracy observed in development may not persist indefinitely.

Imagine a student asks why a neural network initially gives poor answers and improves after training. One analogy is a musician repeatedly practicing a piece, evaluating mistakes, and adjusting technique. The important difference is that the network adjusts numbers in a mathematical function, not conscious understanding. The network first predicts using its current weights, computes loss, uses backpropagation to identify gradients, and then changes weights to reduce future loss. A useful study question is: what is the difference between backpropagation and the optimizer? Backpropagation calculates the gradients; the optimizer uses those gradients to update parameters. Another useful distinction is between a parameter and a hyperparameter: weights and biases are learned from data, whereas learning rate, batch size, and architecture are generally chosen through model design and validation.

03 | Computer Vision and Convolutional Neural Networks

Computer vision concerns methods that enable computers to extract information from images or video. Applications include identifying defects in manufactured goods, reading handwritten text, recognizing road signs, and tracking moving objects. A digital image is represented as an array of pixel values, often with red, green, and blue channels. Yet a raw pixel array does not directly tell a system where an object begins or what it means. Computer vision algorithms must handle variations in lighting, viewpoint, image resolution, background clutter, occlusion, and motion. Common tasks include image classification, object detection, semantic segmentation, instance segmentation, keypoint estimation, and optical character recognition. Each task produces a different kind of output, so matching the model to the question matters.

In image classification, the model assigns a label to an entire image, such as 'cat' or 'dog.' Object detection identifies both object classes and bounding boxes indicating their approximate locations. Semantic segmentation predicts a class for each pixel; instance segmentation goes further by separating individual objects of the same class. For example, a traffic-camera application might classify a scene as busy, detect all visible cars with rectangles, or assign a separate pixel mask to every vehicle. These outputs are related but not interchangeable. A bounding box is fast to annotate and analyze, while a detailed segmentation mask is needed when an application requires the precise shape or area of a visible region.

Convolutional neural networks, or CNNs, became important because their operations exploit local spatial structure. A convolutional filter slides across the image and computes weighted combinations of nearby pixels. The same filter weights are reused across image locations, a property called parameter sharing. Early filters can respond to edges, textures, and color changes; deeper layers can combine local patterns into more abstract features. Strides and pooling layers reduce spatial resolution, while padding controls the treatment of image borders. A typical CNN contains convolution layers, nonlinear activations, occasional downsampling, and a final task-specific output head. Parameter sharing often makes CNNs more efficient than treating every pixel as an unrelated input to a fully connected network.

Training a vision model requires representative images and reliable annotations. Image augmentation creates plausible transformed examples, such as small rotations, crops, horizontal flips, or brightness adjustments. Augmentation can help a model generalize, but it must respect the task: horizontally flipping a medical image or a written digit may change clinically or semantically important information. Transfer learning is useful when labeled data is limited. A model pretrained on a large image collection can be adapted by replacing and fine-tuning its output layers. Fine-tuning all layers may provide further gains, although it needs more data and careful learning rates. Train, validation, and test splits should avoid near-duplicate images from the same person, video, or capture session appearing across partitions.

Feature-based approaches are also relevant. Harris corner detection identifies locations where image intensity changes strongly in two directions. SIFT detects scale- and rotation-tolerant keypoints and builds local feature descriptors. ORB combines fast keypoint detection with compact binary descriptors, making it useful when computation is constrained. A brute-force matcher compares descriptors between images, and matching quality can be improved through distance checks and geometric consistency tests. These traditional methods remain valuable for image stitching, alignment, and low-data tasks. A

CNN and a keypoint matcher solve different kinds of problems: a CNN may classify objects, while feature matching can establish correspondence between views of the same scene.

Evaluation depends on the task. Classification uses accuracy, precision, recall, and confusion matrices. Detection commonly uses intersection over union, or IoU, to compare predicted and true bounding boxes, and average precision to summarize precision-recall behavior across confidence thresholds. Segmentation also uses IoU or Dice similarity to compare predicted pixel regions with annotations. Real-time applications additionally care about inference latency, memory footprint, and processing frames per second. A system with high benchmark accuracy may still fail in poor lighting or unfamiliar environments. Careful error analysis should test changes in camera angle, device, background, and demographic or environmental conditions relevant to the application.

For a concrete example, suppose an astronomy learning tool accepts a night-sky photograph. The computer vision component might detect bright point-like sources, estimate their positions, and compare spatial relationships to a reference star catalog. Merely detecting bright points is not enough to label a constellation confidently: planets, satellites, image noise, lens artifacts, and city lights can produce confusing signals. Additional metadata, geometric matching, and confidence checks are required. Similarly, identifying flames in an image is different from measuring the physical temperature or predicting fire spread. A practical lesson is that computer vision models recognize patterns in visual data, but their outputs require domain context and error handling before being treated as real-world conclusions.

04 | Natural Language Processing and Word Embeddings

Natural language processing, or NLP, studies how computational systems represent, analyze, and generate human language. Language is difficult because meaning depends on context, grammar, world knowledge, speaker intention, and cultural conventions. The word 'bank,' for instance, can refer to a financial institution or the side of a river. The sentence 'I saw her duck' could describe an animal or a movement. Simple keyword matching often misses such distinctions. NLP systems address tasks including document classification, named-entity recognition, sentiment analysis, question answering, translation, summarization, information extraction, and dialogue. These tasks differ in the type of supervision required and the extent to which they depend on context outside the text.

A traditional NLP pipeline might begin with sentence splitting, tokenization, normalization, and feature extraction. Tokenization divides text into smaller units such as words, subwords, or characters. Normalization may standardize case or spelling when that is appropriate. Bag-of-words represents a document by word counts, ignoring word order. Term frequency-inverse document frequency, or TF-IDF, reduces the importance of terms that appear everywhere and emphasizes terms that are relatively distinctive. These representations are useful for efficient search and classification, but they treat closely related words as unrelated unless they share surface forms. They also struggle to represent the difference between 'the dog chased the cat' and 'the cat chased the dog' when the bag of words is identical.

Word embeddings place language units into numerical vectors, ideally making some linguistic or semantic relationships easier to model. Word2Vec and GloVe are familiar examples of approaches that learn one vector per word type. With a static embedding, the vector assigned to 'bank' remains the same in every sentence, which can be limiting. Contextual language models address this by computing representations that depend on surrounding words. The embedding for 'bank' in 'the fisherman sat by the bank' can then differ from the embedding in 'the bank approved the loan.' Sentence embeddings go further by representing sentences or short documents as vectors, enabling semantic similarity comparisons and clustering of passages by meaning.

Similarity between embeddings is often measured using cosine similarity. For vectors a and b, cosine similarity divides their dot product by the product of their magnitudes. The result indicates how aligned the vectors are in geometric space. This can help retrieve a paragraph about 'neural network optimization' for a question asking 'how weights are updated,' even when the wording is different. However, high embedding similarity does not prove factual correctness. Two passages can discuss the same topic while making contradictory claims. Embedding models may also perform differently across languages and specialized fields. Their usefulness depends on how well the model's training data and objectives match the application.

Modern NLP models frequently use subword tokenization, allowing rare words to be represented as combinations of familiar pieces. This reduces the need for a complete dictionary of every possible word, although unusual technical terms can still be split awkwardly. Transformer-based models capture relationships between tokens using attention mechanisms. Encoder models are often useful for understanding-oriented tasks such as classification or extracting information, whereas decoder models are especially common for generating the next token in a sequence. Encoder-decoder models can transform input text into output text, as in translation or summarization. The terminology describes

architectural patterns, not an absolute separation of capabilities.

Evaluating NLP systems is challenging because multiple outputs may be valid. Classification tasks can use precision and recall, but summarization may require factual consistency, coverage, readability, and human review. Token-overlap metrics may reward a summary that repeats source wording while missing the meaning. For retrieval, precision at k, recall at k, mean reciprocal rank, and normalized discounted cumulative gain measure different aspects of ranking quality. In question answering, it is useful to distinguish answers supported by the source from plausible but unsupported responses. A tutoring assistant should sometimes say the uploaded notes do not answer a question rather than inventing an explanation that appears authoritative.

Imagine a student uploads a physics textbook and asks, 'What causes a rainbow?' The question contains the words 'causes' and 'rainbow,' but a relevant passage might instead describe refraction, reflection, and dispersion of sunlight in raindrops. Keyword retrieval can work if those terms occur together, while embedding-based retrieval may find the passage based on semantic meaning. The language model can then explain the concept in accessible wording. This example shows why NLP applications often combine several methods: tokenization processes the text, retrieval finds candidate passages, contextual modeling interprets the question, and generation creates a readable answer. The final step still requires checking whether the answer is genuinely supported by the selected evidence.

05 | Transformers, Attention, and Large Language Models

The Transformer is a neural-network architecture introduced to model relationships among tokens using attention rather than relying only on recurrent computation. In a traditional recurrent neural network, information flows step by step through a sequence, which can make long-range dependencies and parallel training difficult. A Transformer processes collections of token representations and allows each token to selectively incorporate information from other positions. Attention makes it possible for the representation of a word such as 'it' to depend on a relevant earlier noun. Modern large language models, or LLMs, commonly rely on Transformer components and are trained on extensive text collections to predict tokens or perform related language objectives. Their abilities emerge from statistical learning rather than from a database of perfectly verified facts.

Before tokens enter a Transformer, a tokenizer converts text into token IDs. An embedding layer maps each token ID to a learned vector. Because a sequence also has order, the model needs positional information. Sinusoidal positional encodings, learned positional embeddings, and rotary positional methods are different ways of giving attention mechanisms information about token position. Removing positional information would make it harder to distinguish sequences containing the same tokens in different orders. For example, 'the teacher praised the student' and 'the student praised the teacher' involve similar token identities but different relationships. The architecture must preserve enough ordering information to represent this distinction.

The central attention calculation uses three projected vectors called queries, keys, and values. A token's query is compared with the keys of available tokens to obtain compatibility scores. These scores are scaled and normalized by a softmax operation, producing attention weights. A weighted combination of value vectors becomes the attention output. In scaled dot-product attention, the query-key score is divided by the square root of the key dimension to limit how large the scores become. Multi-head attention repeats this operation across several learned projection spaces. Different heads can capture different relationships, though it is a mistake to assume every head has a single simple linguistic interpretation.

Transformer blocks also contain feedforward networks, residual connections, and normalization layers. Self-attention lets positions in one sequence interact with each other. Cross-attention lets a decoder condition on representations produced by another component, such as an encoder. An encoder-only architecture can use bidirectional context and is often adapted to classification, tagging, or retrieval embeddings. Decoder-only architectures generally use causal attention masks so a position cannot attend to future tokens during ordinary next-token training. Encoder-decoder architectures combine an input representation stage with an output generation stage. BERT is an important encoder-oriented model family, GPT illustrates decoder-oriented language modeling, and BART or T5 are examples of encoder-decoder designs.

A large language model usually learns from a pretraining objective on a diverse dataset. Instruction tuning can subsequently train it to respond to commands and conversational requests. Additional preference-oriented training may influence helpfulness and response style. During generation, the model produces a probability distribution over possible next tokens. Greedy decoding selects the highest-probability token at each step, while sampling introduces controlled variation. Temperature modifies how concentrated the token probabilities are: lower temperatures generally make sampling

more predictable, while higher temperatures permit more diverse outputs. Temperature is not a numerical measure of truth. A low-temperature model can still confidently generate an incorrect statement.

LLMs have several limitations. They can hallucinate, meaning they generate fluent text that lacks grounding in reliable evidence. Their context windows are finite, even if large, and long inputs can be expensive or difficult to use effectively. They may also reproduce biases present in training data, misunderstand unusual prompts, or follow distracting instructions contained inside retrieved documents. Retrieval-augmented generation can improve evidence availability, but retrieved text must be treated as data rather than trusted instructions. For high-stakes uses, developers need evaluation sets, systematic error analysis, human oversight, privacy controls, and safeguards against exposing private information through prompts or logs.

A useful educational example is a Transformer answering, 'Why does a larger image patch size reduce the computational cost of a Vision Transformer?' For a 224-by-224 image, 16-by-16 patches produce 196 patches, while 32-by-32 patches produce 49 patches. Standard attention compares token positions with one another, so its attention-score matrix grows roughly with the square of sequence length. The shorter sequence can therefore reduce attention-related computation, although it may also lose fine spatial detail. The example connects Transformers to image processing and illustrates why tokenization decisions matter. A student should remember that attention is a learned weighting mechanism, positional encoding provides ordering or spatial information, and generation quality depends on training, prompting, retrieval context, and evaluation rather than attention alone.

06 | Reinforcement Learning and Adaptive Tutoring

Reinforcement learning, or RL, studies how an agent should choose actions while interacting with an environment to maximize expected cumulative reward. Unlike ordinary supervised learning, an RL agent is not simply told the correct action for every possible situation. It observes a state, chooses an action, receives a reward, and reaches a new state. This interaction can be modeled as a Markov decision process, or MDP, with states, actions, transition dynamics, rewards, and a discount factor. The agent's policy specifies which action to choose in each situation. In adaptive tutoring, the environment can be viewed as a student's evolving learning process, while the agent chooses a teaching action such as a question difficulty or a worked example.

A state should summarize information relevant to future decisions. For a tutoring example, useful components might include estimated concept mastery, previous question difficulty, recent correctness, and how often the student has requested hints. The exact state representation is a design decision: a small discrete state enables an understandable Q-table but may discard details, while a high-dimensional continuous representation offers flexibility at the cost of more complex learning. An action might be to ask an easy question, ask a harder question, revisit a topic, or present a worked example. Rewards must reflect the real instructional objective. Rewarding correctness alone is dangerous because an agent might learn to ask only trivial questions to collect easy points.

The exploration-exploitation dilemma is central to RL. Exploitation selects an action currently believed to be good; exploration tests actions that may turn out to be better. An epsilon-greedy policy chooses a random action with probability epsilon and otherwise chooses an action with the best current estimate. If epsilon equals one, the agent explores all the time. If epsilon becomes very small too soon, the agent can become stuck favoring a suboptimal action due to limited early evidence. Other exploration strategies include upper confidence bound methods, Thompson sampling, and softmax sampling based on action values. In a tutoring application, exploration should be constrained by educational safety rules rather than allowing arbitrarily frustrating or inappropriate actions.

Q-learning is a model-free, off-policy temporal-difference control method. It updates an action-value estimate using an observed reward plus the best estimated future action value. Its familiar update is Q(s,a) equals Q(s,a) plus alpha times the difference between reward plus gamma times max Q(s',a') and the old Q(s,a). Alpha is the learning rate; gamma discounts future rewards. Q-learning is called off-policy because its learning target uses the best next-action value even when the behavior policy explores other actions. SARSA is a related on-policy method that updates using the value of the next action actually selected by the behavior policy. Both methods can update after each transition without waiting for an episode to finish.

Monte Carlo methods instead estimate values from completed episodes by averaging observed returns. First-visit Monte Carlo uses the first occurrence of a state in an episode, while every-visit Monte Carlo uses each occurrence. Monte Carlo prediction does not require transition probabilities but typically depends on finite completed episodes. Temporal-difference methods bootstrap from current value estimates, which can support continuing tasks where episodes do not naturally terminate. Dynamic programming, by contrast, generally assumes access to the environment's transition and reward model. Policy iteration alternates policy evaluation with policy improvement, while value iteration repeatedly applies an optimality backup before deriving a policy. These distinctions help explain why different RL

algorithms suit different settings.

A practical tutoring system must estimate the student's state, because knowledge itself cannot be observed directly. Bayesian Knowledge Tracing, or BKT, uses probabilities to estimate concept mastery from answers. Its parameters include initial mastery, learning probability, slip probability, and guess probability. Slip models an incorrect response despite knowing the material; guess models a correct response without knowing it. An RL policy can use the BKT mastery estimate as one state component. For example, a student with low mastery and multiple recent mistakes might receive a lower-difficulty question or a guided example. A student with consistently strong responses might receive a more demanding task. This is a policy choice informed by a probabilistic model, not a guarantee of student understanding.

Evaluation of tutoring RL requires special care. Simulation permits fast comparisons with fixed, random, or heuristic baselines, but simulated students are defined by assumptions and may not behave like real learners. Important metrics can include learning gain, completion time, frustration, hint usage, and worked-example frequency. Off-policy evaluation tries to estimate a candidate policy's value from logged interactions collected under another policy. Importance weighting and doubly robust estimators can help, but they become unreliable if the new policy chooses actions rarely seen in the logs. A safe classroom deployment also needs action constraints, privacy protections, student consent where applicable, and human review. An RL tutor is successful not merely when its numerical reward rises, but when real learners receive effective, fair, understandable instruction.

07 | Retrieval-Augmented Generation and Hybrid Search

Retrieval-augmented generation, or RAG, is a design pattern that combines information retrieval with a generative language model. Instead of requiring a model to answer solely from what it learned during training, the application retrieves potentially relevant material from a specified collection and places that evidence in the model's working context. RAG is useful for textbooks, internal manuals, class notes, product documentation, and other sources that an assistant should consult directly. In a student-tutoring scenario, a learner might upload lecture notes and ask for the definition of a Markov decision process. The system should locate the relevant passage and answer using its contents, ideally with a reference to the source. The purpose is evidence-grounded assistance, not a guarantee that all hallucinations disappear.

A typical RAG pipeline has an ingestion stage and a query-time stage. During ingestion, the application accepts PDF, text, or other documents, extracts readable content, and records metadata such as filename, section, and page number when available. Text is split into smaller chunks because retrieving entire long documents is expensive and often produces too much unrelated context. Chunk size and overlap are design parameters. Small chunks may be precise but lose explanatory context; large chunks preserve context but may dilute relevance. Overlap repeats a little material across adjacent chunks so a definition spanning a boundary remains available. For example, a sample tutor corpus could split notes into 180-word chunks with 30-word overlap; those numbers are illustrative application choices, not universal best practices.

One retrieval strategy is lexical search using BM25. BM25 scores documents according to how well query terms match terms in candidate passages, with adjustments for term frequency and document length. It is helpful for exact terminology, equations, identifiers, and unusual abbreviations. If the question contains 'Bellman equation,' a passage containing that exact phrase will often be a strong candidate. However, lexical retrieval can miss conceptually relevant writing when the vocabulary differs. An embedding model maps queries and passages into numerical vectors; vector similarity can identify passages about the same ideas even when the words are different. This is often called dense or semantic retrieval. Dense retrieval adds flexibility but can also return semantically related passages that do not actually contain the answer.

Hybrid retrieval combines lexical and dense results instead of relying on a single ranking method. The two score scales are not automatically comparable, so developers often use rank-based fusion. Reciprocal Rank Fusion, or RRF, awards each passage a score based on its positions in the separate ranked lists. A familiar form adds one divided by k plus rank, using a constant k such as 60. Candidate results from hybrid search can then be passed to a reranker. A cross-encoder evaluates the query and each candidate passage together, usually at greater computational cost than a separate embedding lookup. The reranker tries to put the most relevant passage near the top, so the language model sees high-quality evidence within the available context budget.

Retrieval quality should be evaluated separately from answer quality. Retrieval recall at k measures whether relevant evidence appears among the first k results. Precision at k measures the relevance of returned results. Mean reciprocal rank is especially useful when one correct passage is sought, whereas nDCG can reward several relevant passages with different relevance grades. Answer evaluation asks whether the generated response is correct, sufficiently complete, and supported by the retrieved

passages. A fluent answer that contradicts the notes is a grounding failure even when the retrieved passage is relevant. Conversely, a weak retrieval stage can prevent a well-instructed model from finding the information it needs. Logging queries, retrieved identifiers, and evidence is useful for debugging these failures.

RAG systems also face operational issues. PDF extraction may scramble multi-column layouts, tables, mathematical notation, or scanned images. Semantic models consume memory and may not be available on low-resource devices. Retrieval can surface duplicate chunks or conflicting information from different documents. Long context windows cost more and may increase distraction. Prompt injection is another risk: a retrieved document might contain instructions telling the assistant to ignore the user, reveal secrets, or change its behavior. A secure design treats retrieved text as untrusted source material, never as an authority over the application's instructions. Sensitive uploaded materials also require careful storage, access controls, and data-retention policies.

Consider a concrete query: 'Explain why Q-learning is off-policy using my RL notes.' BM25 may retrieve passages containing 'Q-learning' and 'off-policy,' while embeddings may identify a paragraph about learning from the maximum estimated next-action value. Fusion ranks the candidates, and a reranker chooses the most explanatory material. The generative model then explains that the Q-learning target uses max over next-state action values, whereas the behavior policy may have chosen an exploratory action. The system should cite or label the passages it used. A good follow-up test asks a question not covered by the uploaded notes; the tutor should acknowledge missing source support rather than silently treating its own prior knowledge as if it came from the file. In educational applications, this distinction makes RAG both a learning aid and a tool for evaluating trustworthy AI behavior.

08 | Generative AI: GANs, VAEs, and Diffusion Models

Generative artificial intelligence creates new outputs that resemble patterns learned from training data. Its outputs can include text, images, audio, video, code, and structured sequences. Generative models differ from traditional discriminative classifiers. A discriminative classifier might decide whether an image contains a dog, while a generative model might synthesize a new dog image. Generation does not mean that the system has independently invented every visual or linguistic idea; it models statistical structure from examples and produces samples under the learned distribution. The main generative model families include autoregressive models, variational autoencoders, generative adversarial networks, and diffusion models. Each family uses a different objective and sampling procedure.

Generative adversarial networks, or GANs, train two models with competing objectives. The generator produces synthetic examples, while the discriminator attempts to distinguish genuine training examples from generated ones. During learning, the generator improves by finding outputs that the discriminator judges to be real. The adversarial setup can produce sharp, convincing images, but training may be unstable. One common problem is mode collapse, in which the generator repeatedly produces a limited variety of examples rather than covering the diversity of the data. The discriminator can also become too strong, providing weak feedback to the generator. GAN variants have been used for image translation, super-resolution, and data synthesis, but evaluation must consider both realism and diversity.

A variational autoencoder, or VAE, has an encoder and a decoder. The encoder converts an input into parameters of a probability distribution over latent variables, often a mean and variance. A latent sample is then decoded to reconstruct an example. Training balances a reconstruction term, which rewards fidelity to the input, with a regularization term commonly expressed as Kullback-Leibler divergence to a prior distribution. The regularization encourages a structured latent space from which new samples can be drawn. VAEs can support interpolation between examples, representation learning, and anomaly detection. Their samples may look smoother or blurrier than those from some competing image generators, depending on architecture and reconstruction assumptions.

Diffusion models learn to reverse a corruption process. In a common formulation, the forward process gradually adds noise to training examples, and a neural network is trained to predict the noise or an equivalent quantity at different noise levels. During generation, the system starts from random noise and iteratively denoises it into a sample. This process can produce high-quality and diverse images. Text-to-image diffusion systems condition their denoising network on a text representation so that generated images follow a prompt. Classifier-free guidance is one technique that adjusts the strength of conditioning, with tradeoffs between prompt adherence and sample diversity. More denoising steps often increase computation, although modern samplers can generate usable results in fewer steps.

Autoregressive models generate a sequence one element at a time. Many large language models are autoregressive: given a context, they estimate a probability distribution over the next token, select a token according to a decoding strategy, append it, and repeat. An image can also be represented as tokens or patches for autoregressive generation. Unlike GAN generation, which may produce a whole latent-to-image output in one forward path, autoregressive generation explicitly models sequential dependence. Different families therefore have different speed, quality, control, and training properties. Choosing among them depends on whether the application needs continuous images, discrete sequences, editing capability, or structured latent representations.

Generative-model evaluation is more complicated than simply checking accuracy. Images may be assessed for realism, diversity, alignment with prompts, and whether important objects are represented correctly. Metrics such as Fréchet Inception Distance attempt to compare distributions of image features, but a single score cannot capture every desirable property. Human evaluation can identify obvious mistakes, yet human judgments may vary and need consistent procedures. Text generation also requires factuality, relevance, readability, and safety checks. Synthetic outputs can reflect bias in their source data or reveal memorized private details in poorly designed systems. Generated media should not be presented as genuine evidence of events that did not occur.

A classroom example is asking an AI system to generate several illustrations of a solar system for a lesson. A diffusion model may create attractive images, but the positions, sizes, and physical relationships of planets could be scientifically inaccurate. A language model could explain the differences between the images and a correct diagram, and an instructor could check whether the visual examples support learning goals. Another example is generating synthetic training data for rare categories: although it can enlarge a dataset, synthetic data may reproduce the original dataset's biases or fail to represent important edge cases. The key lesson is that generative AI excels at producing plausible samples but still needs domain-aware verification, attribution when appropriate, and careful consideration of what its outputs can legitimately demonstrate.

09 | Responsible AI, Fairness, and Explainability

Responsible artificial intelligence is the practice of designing and operating AI systems in ways that respect people, safety, privacy, accountability, and legitimate social expectations. A model can achieve high accuracy on a benchmark and still produce harmful outcomes if its data is unrepresentative, its outputs are misunderstood, or people cannot challenge its decisions. Responsible AI therefore extends across the entire lifecycle: task definition, data collection, labeling, modeling, evaluation, deployment, monitoring, and retirement. It is not a final checklist attached after training. In education, responsible design matters because an adaptive tutor may influence which questions students receive and what the system believes they know, potentially shaping motivation and access to learning opportunities.

Fairness refers to several related but nonidentical goals. One concern is whether the system performs comparably across relevant populations. Another is whether similar individuals receive similar treatment when differences are not justified by the task. Group metrics such as demographic parity, equalized odds, or equal opportunity formalize different criteria, but these criteria can conflict depending on the data distribution. There is no single fairness number that solves every ethical problem. In a tutoring context, an algorithm should not systematically assign harder questions to one group because its training dataset contains biased performance patterns. It should also be tested across language proficiency, accessibility needs, device types, and different educational backgrounds where those factors affect experience.

Privacy begins with collecting only the information genuinely needed for the task. A tutoring system might need question outcomes and concept mastery, but not a student's precise location or unrelated personal records. Data minimization, access control, encryption where appropriate, deletion policies, and clear disclosures help limit risk. Training logs can be sensitive because they reveal weaknesses, learning difficulties, or personal writing. When using an external generative model provider, developers should understand what text is sent to that provider and what retention or processing rules apply. Uploading lecture notes also raises copyright and confidentiality questions. A secure application should not expose one student's documents or chat history to another student's session.

Explainability seeks to help people understand how and why a model produced an output. A linear model can often present its coefficients, while a decision tree can show a path of splits. A tabular Q-learning tutor can display its estimated student state, available teaching actions, Q-values, and the probability of the selected action. This is more informative than showing only 'AI chose Level 3.' However, a probability of selecting an action does not measure the probability that the action was objectively correct. Post-hoc explanation methods such as SHAP or LIME approximate feature contributions for a model's prediction. Such explanations can be useful for debugging and communication, but they should not be mistaken for definitive causal explanations of how a complex model reasons.

Robustness asks whether a system remains reliable when inputs are noisy, incomplete, or different from training data. A computer vision classifier may fail under unexpected lighting; a language model may be misled by adversarial instructions hidden in a document; a tutoring policy may behave badly for a student whose learning pattern is unlike any simulated learner. Monitoring should therefore track performance drift, error categories, and harmful outputs after deployment. Human oversight is especially important when decisions have serious consequences. In education, AI-generated explanations should

support teachers and students rather than replace all professional judgment about learner needs.

Transparency includes communicating model limitations and what has actually been measured. A system trained against simulated students must not describe those results as proven learning gains in real classrooms. If an RAG answer was generated using three retrieved chunks, that fact can be reported, but it should not be presented as a 100-percent factual guarantee. Confidence scores, similarity scores, and action probabilities are different quantities and should be labeled precisely. Evaluation documents should record dataset versions, preprocessing choices, hyperparameters, and known failures. Model cards or similar documentation can summarize intended uses, excluded uses, training conditions, and important evaluation limitations.

Consider a hypothetical learning assistant that marks a learner as weak in algebra after two incorrect responses. Responsible design would ask whether the questions were clearly worded, whether the student received an accessibility accommodation, whether grading was correct, and how uncertain the mastery estimate is. The assistant should allow additional practice and correction instead of permanently labeling the student. The same principle applies to a generative model that answers confidently without source evidence: the interface should expose uncertainty or evidence gaps rather than encouraging blind trust. Responsible AI is not opposed to model performance; it defines a broader standard in which usefulness, accuracy, fairness, transparency, privacy, and human control all matter.

10 | AI Agents, Planning, Tools, and Evaluation

An AI agent is a system that observes information, selects actions, and uses tools or internal reasoning processes to pursue a goal. The term can describe anything from a simple rule-based controller to a language-model application that plans, searches documents, calls software tools, and revises its work. Not every chatbot is an autonomous agent: a system that generates one reply without managing tools or a task workflow is better described as a language model interface. Agentic design becomes useful when a task requires several coordinated steps, such as reading source documents, identifying missing facts, computing a result, drafting an explanation, and checking that the explanation is grounded in the available evidence.

Planning decomposes a goal into smaller operations. Suppose a student asks an AI tutor to explain gradient descent using their notes, generate practice questions, and prepare a revision schedule. A planner can identify three distinct activities: retrieve relevant passages, produce a lesson with questions, and use tracked mastery to allocate study time. A tool-using agent may then call a retrieval service, a quiz generator, and an analytics module. The agent should know the capabilities and limits of each tool. A retrieval result is evidence, not an instruction to ignore safeguards. A calculator can execute arithmetic, but the agent remains responsible for deciding whether the calculation was appropriate for the question.

Multi-agent systems distribute subtasks among specialized roles. In a teaching workflow, a Planner might choose an explanation strategy, a Retriever might fetch note passages, a Teacher might draft the lesson, and a Critic might check whether the result follows the instructions and cites valid sources. This arrangement can improve organization and enable explicit review stages. However, different roles do not guarantee independent expertise when all agents use the same underlying model. A critic can miss the same mistake as the teacher, and extra model calls increase latency and cost. Structured intermediate outputs, clear acceptance criteria, and logging help developers understand where a failure occurred.

A common agent control loop is observe, plan, act, check, and update. The observation may include a user request and current session state. Planning identifies an action sequence. Acting invokes a tool or generates an intermediate artifact. Checking evaluates whether the action succeeded and whether requirements are satisfied. Updating revises the task state before continuing or producing a final reply. Loop limits matter because an unconstrained agent might repeat failed tool calls indefinitely. Developers can impose a maximum number of iterations, time or cost budgets, and stop conditions. Human approval may be appropriate before taking actions that change external records, send messages, or otherwise produce consequences outside a draft response.

Agent memory is another important design choice. Short-term working memory consists of the current prompt, active tool outputs, and recent dialogue. Long-term memory stores selected information across sessions, such as preferences or progress summaries, subject to privacy controls. Retrieval from a vector database or document index is not automatically the same thing as personal memory: retrieved notes are reference material, while stored student progress describes previous interactions. Poor memory design can create contradictions or expose private data. A learning assistant should distinguish current evidence from old estimates, and should update or delete stored information according to authorized user actions and clear retention rules.

Evaluating an agent requires more than judging the fluency of its final response. A task-success metric asks whether the requested outcome was achieved. Tool-use evaluation checks whether the agent selected the appropriate tool and interpreted the result correctly. Grounding evaluation checks whether factual claims are supported by available source passages. Other useful metrics include latency, number of tool calls, failure recovery, cost per successful task, and harmful or unauthorized actions. Multi-agent workflows need step-level traces because an incorrect final answer may result from a bad retrieval query, irrelevant evidence, flawed generation, or an inadequate critic check. Reproducible tests should include ordinary cases, ambiguous inputs, missing source information, and deliberate instruction attacks embedded in documents.

For example, imagine an AI teaching assistant receives the request, 'Using my uploaded notes, explain temporal-difference learning and create three revision questions.' The Planner decides that it must first retrieve the definition and update rule. The Retriever finds passages on bootstrapping, Q-learning, and Monte Carlo comparison. The Teacher writes a concise explanation and generates questions at different difficulty levels. The Critic checks that the update rule matches the evidence, that three questions were produced, and that their stated answers are internally consistent. If the retrieved notes contain no information about temporal-difference methods, the appropriate output is an explicit evidence limitation, not a fabricated citation. The lesson illustrates the main promise and limitation of agents: they can coordinate useful workflows, but reliability comes from good tools, bounded actions, transparent evidence, and systematic evaluation rather than from the word 'agent' itself.